



Tecnológico Nacional de México

Campus Chilpancingo



Ingeniería en sistemas Computacionales

Materia:

Inteligencia Artificial

ENTRENAMIENTO DE IMÁGENES

YOLO(etiquetado de imágenes)

Alumna :

Sandra Bianey Moctezuma Mónico

Grupo

S8A

Docente:

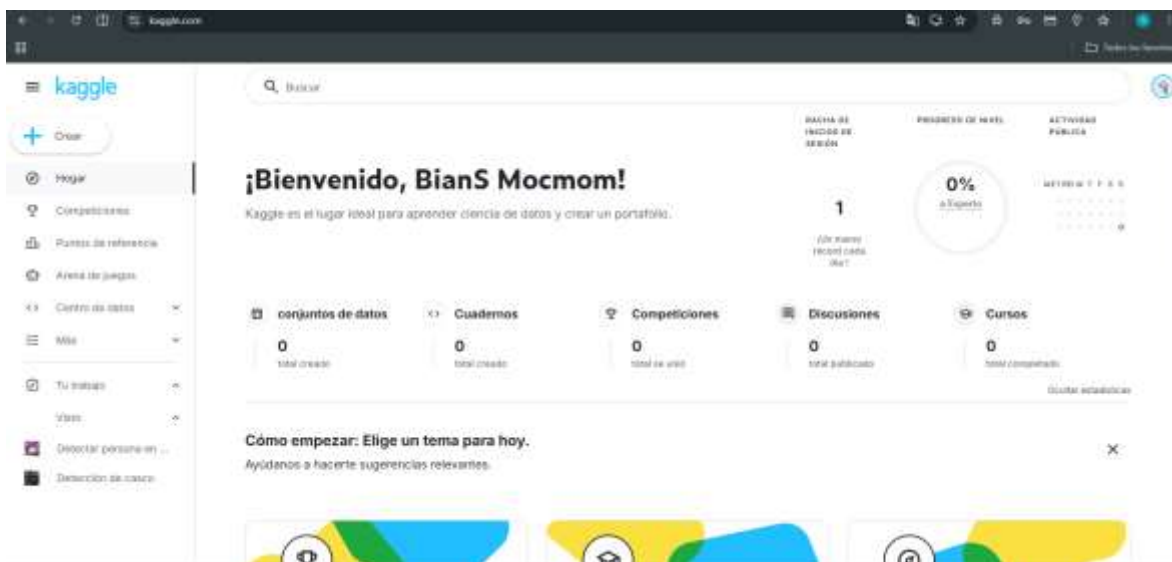
Yanet Evangelista Alcocer

ETIQUETADO Y ENTRENAMIENTO DE IMÁGENES YOLO

Paso 1 – Búsqueda y descarga del dataset en Kaggle

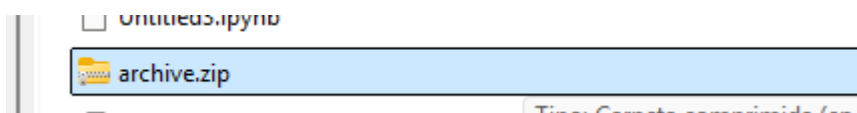
Se accede a la plataforma **Kaggle** (kaggle.com) con el usuario **BianS Mocmom**. Desde aquí se utilizó el buscador para encontrar el dataset de imágenes de motos que se usará para entrenar el modelo **YOLO**.

Una vez localizado el dataset, se procedió a su descarga para usarlo en el entrenamiento.



Paso 2 – Descarga del dataset

Se descargó el dataset desde Kaggle en formato comprimido **archive.zip**, el cual contiene las imágenes de motos que se utilizarán para entrenar el modelo **YOLO**.

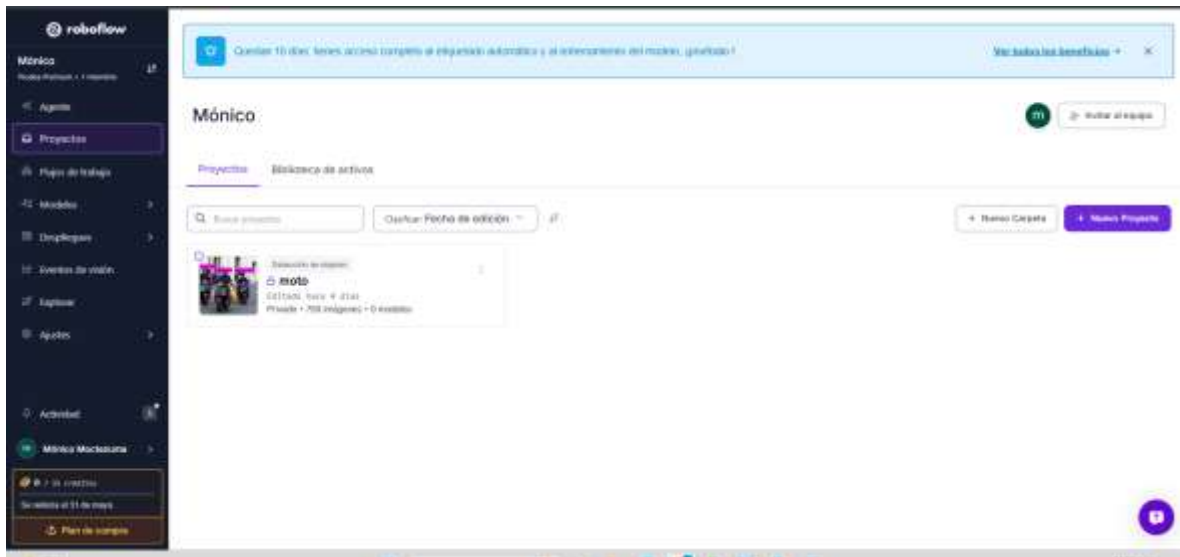


Paso 3 – Carga y etiquetado de imágenes en Roboflow

Las imágenes descargadas del dataset se subieron a la plataforma **Roboflow**, con el usuario **Mónico Moctezuma**. Se creó un proyecto llamado "**moto**" de tipo **Detección de objetos**, el cual cuenta con:

- **780 imágenes** cargadas
- **0 modelos** entrenados hasta el momento
- Editado hace **4 días**

Roboflow se utilizó para **etiquetar (anotar)** las imágenes, es decir, marcar manualmente las motos dentro de cada imagen para que el modelo **YOLO** pueda aprender a detectarlas.



Paso 4 – Configuración del entorno en Google Colab

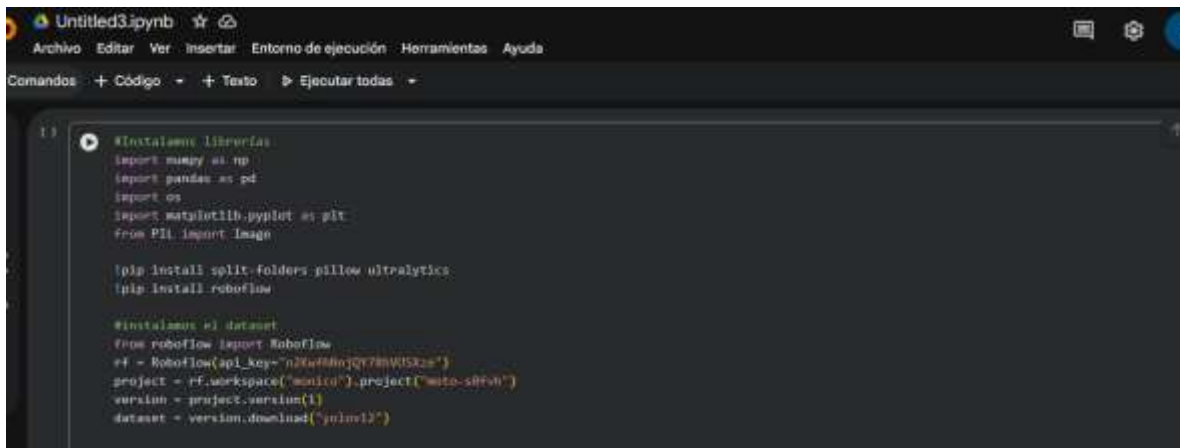
Se abre un cuaderno de **Google Colab** (Untitled3.ipynb) donde se prepara el entorno para el entrenamiento del modelo. El código realiza dos acciones principales:

Instalación de librerías:

- numpy, pandas, os, matplotlib, PIL – librerías básicas de Python
- split-folders, pillow, ultralytics – para manejo de imágenes y YOLO
- roboflow – para conectarse al proyecto de Roboflow

Descarga del dataset desde Roboflow:

- Se conecta al workspace "**monico**"
- Se accede al proyecto "**moto-s0fvh**"
- Se descarga la versión 1 del dataset en formato **YOLOv12**



```
1) #Instalamos librerías
import numpy as np
import pandas as pd
import os
import matplotlib.pyplot as plt
from PIL import Image

!pip install split-folders pillow ultralytics
!pip install roboflow

#Instalamos el dataset
from roboflow import RoboFlow
rf = RoboFlow(api_key="n2Kw4MnjqY78nW5K2s")
project = rf.workspace("monico").project("moto-sfsh")
version = project.version(1)
dataset = version.download("yolov12")
```

Paso 5 – Instalación de librerías y descarga del dataset

Se ejecuta la celda de código y se puede ver en la consola cómo se instalan todas las dependencias necesarias y se descarga el dataset:

Instalación de paquetes:

- Se instalan librerías como roboflow, opencv-python-headless, pillow-avif-plugin, idna, entre otras
- Algunas ya estaban instaladas y se actualizaron automáticamente

Descarga del dataset:

- Se conecta exitosamente al workspace y proyecto de Roboflow
- Se descarga el dataset en formato **YOLOv12** con un total de **33,913 archivos** a una velocidad de ~10,469 it/s
- Se extrae el ZIP con **1,558 archivos** a una velocidad de ~9,504 it/s

Al finalizar, el dataset queda listo y descomprimido en el entorno de Colab para comenzar el entrenamiento.


```
49/50 7.13G 0.5266 0.327 1.03 32 640: 100% 9/9 1.51t/s 8.0s
Class Images Instances Box(P R mAP50 mAP50-95): 100% 2/2 1.1s/1s 2.2s
all 155 174 0.975 0.983 0.991 0.83

Epoch GPU mem box_loss cls_loss dfl_loss Instances Size
50/50 7.13G 0.5823 0.323 1.017 32 640: 100% 9/9 1.51t/s 8.1s
Class Images Instances Box(P R mAP50 mAP50-95): 100% 2/2 1.1s/1s 1.6s
all 155 174 0.988 0.98 0.99 0.899

50 epochs completed in 0.143 hours.
Optimizer stripped from /content/runs/detect/Deteccion motociclistas/Deteccion motociclistas/weights/last.pt, 6.20M
Optimizer stripped from /content/runs/detect/Deteccion motociclistas/Deteccion motociclistas/weights/best.pt, 6.20M

Validating /content/runs/detect/Deteccion motociclistas/Deteccion motociclistas/weights/best.pt...
Ultralytics 8.6.52 Python-3.12.13 torch-2.10.0-cuda12.1 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 3,005,643 parameters, 0 gradients, 8.1 GiB
Class Images Instances Box(P R mAP50 mAP50-95): 100% 2/2 1.1s/1s 2.2s
all 155 174 0.982 0.989 0.994 0.811

Speed: 0.2ms preprocess, 2.6ms inference, 0.8ms loss, 1.5ms postprocess per image
Results saved to /content/runs/detect/Deteccion motociclistas/Deteccion motociclistas
ultralytics.utils.metrics.DetMetrics object with attributes:
ap_class_index: array([0])
box: ultralytics.utils.metrics.Metric object
confusion_matrix: ultralytics.utils.metrics.ConfusionMatrix object at 0x797ec90838b0
curves: {'Precision-Recall(0)', 'F1-Conidence(0)', 'Precision-Conidence(0)', 'Recall-Conidence(0)'}
----- Precision-Recall (0) -----
----- F1-Conidence (0) -----
----- Precision-Conidence (0) -----
----- Recall-Conidence (0) -----
```

Paso 8 – Inferencia con el modelo entrenado

Se carga el mejor modelo guardado y se realiza una prueba de detección sobre una imagen:

Código ejecutado:

- Se carga el modelo entrenado desde weights/best.pt
- Se realiza la inferencia sobre la imagen /content/moto.jpg

Resultados de la inferencia:

- Imagen procesada: moto.jpg (384x640 px)
- Velocidad total: 2.5ms preproceso + 15.6ms inferencia + 1.6ms postproceso
- Resultados guardados en: /content/runs/detect/predict-3

El modelo detectó exitosamente la moto en la imagen de prueba en tan solo 18ms, lo que demuestra que es suficientemente rápido para aplicaciones en tiempo real.

```
#Inferimos con nuestro modelo
modelo_entrenado = YOLO("/content/runs/detect/Deteccion motociclistas/Deteccion motociclistas/weights/best.pt")

# Realizar inferencia en una imagen
resultados = modelo_entrenado.predict(source="/content/moto.jpg", save=True)

Image 1/1 /content/moto.jpg: 384x640 1 0, 15.6ms
Speed: 2.5ms preprocess, 15.6ms inference, 1.6ms postprocess per image at shape (1, 3, 384, 640)
Results saved to /content/runs/detect/predict-3
```

Paso 9 – Resultado de la detección y descarga del modelo

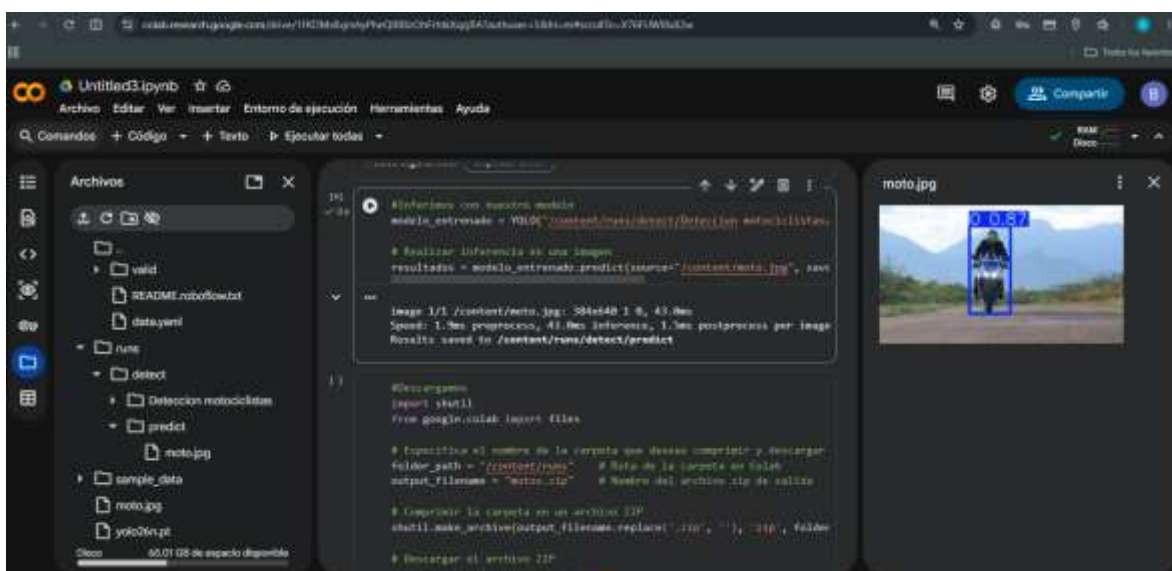
Se puede ver el resultado visual de la inferencia y la preparación para descargar el modelo:

Resultado de la detección: En la vista previa de moto.jpg se observa una motocicleta detectada correctamente con un recuadro azul y una confianza de **0.87 (87%)**.

Estructura de archivos generada:

- Carpeta runs/detect/Deteccion motociclistas/ con los resultados
- Carpeta predict/ con la imagen moto.jpg procesada
- Archivos data.yaml y README.roboflow.txt del dataset

Descarga del modelo: Se ejecuta un código para comprimir la carpeta runs/ en un archivo motos.zip y descargarlo desde Google Colab al equipo local, usando las librerías shutil y google.colab.files.



Paso 10 – Compresión y descarga del modelo entrenado

Se ejecuta el código final para guardar y descargar los resultados del entrenamiento:

- Se importan las librerías shutil y files de Google Colab
- Se especifica la carpeta /content/runs como origen, que contiene el modelo entrenado y los resultados
- Se comprime toda la carpeta en un archivo llamado motos.zip
- Se descarga el archivo motos.zip al equipo local con files.download()

Esto permite conservar el modelo entrenado (best.pt) y todos los resultados fuera de Google Colab, ya que el entorno de Colab se elimina al cerrar la sesión.

```
#Descargamos
import shutil
from google.colab import files

# Especifica el nombre de la carpeta que deseas comprimir y descargar
folder_path = "/content/runs"      # Ruta de la carpeta en Colab
output_filename = "motos.zip"      # Nombre del archivo zip de salida

# Comprimir la carpeta en un archivo ZIP
shutil.make_archive(output_filename.replace('.zip', ''), 'zip', folder

# Descargar el archivo ZIP
files.download(output_filename)
```

Paso 11 – Archivo descargado exitosamente

Se confirma que el archivo motos.zip fue descargado correctamente al equipo local, el cual contiene el modelo entrenado y todos los resultados generados durante el proceso.

